Reprodução de uma Formulação de Programação Linear Inteira Mista para o Sequenciamento de Recuperadoras em Pátios de Minério Visando à Minimização do Atraso Total

João Paulo da Cunha Faria
Engenharia de Computação
Centro Federal de Educação Tecnológica de Minas Gerais
Divinópolis, Minas Gerais, Brasil
joao@aluno.cefetmg.br

***Resumo*—Os pátios de estocagem são componentes essenciais da cadeia logística da mineração. O sequenciamento das recuperadoras que retomam o minério das pilhas configura um problema de escalonamento em máquinas paralelas não-relacionadas, com tempos de preparação dependentes da sequência, datas de entrega e elegibilidade de máquinas. Este trabalho reproduz a formulação de programação linear inteira mista (MILP) proposta por Florinda, Souza e Cota (2024) para minimizar o atraso total ponderado. A formulação foi reimplementada com a API do resolvedor Gurobi em linguagem Python e avaliada sobre um conjunto de 30 instâncias de pequeno porte. Nos experimentos, o resolvedor encontrou a solução ótima para todas as 30 instâncias, com destaque para o grupo de 12 tarefas como o de maior esforço computacional, reproduzindo qualitativamente o comportamento reportado no trabalho original.**

***Index Terms*—Sequenciamento de recuperadoras, atraso total, mineração, programação linear inteira mista, Gurobi.**

I. Introdução

Os pátios de estocagem constituem componentes essenciais da cadeia logística da mineração, influenciando de forma significativa o seu desempenho. Após o beneficiamento, o minério é encaminhado por correias transportadoras até os pátios, onde é organizado em pilhas para facilitar o desaguamento parcial e absorver as flutuações de demanda no transporte e na produção. Nesses pátios, utilizam-se *recuperadoras* para retomar o minério das pilhas e transferi-lo para as correias transportadoras, conduzindo-o a outros locais, como carregadores de navios.

Uma programação eficaz das recuperadoras — também conhecida como sequenciamento — pode contribuir significativamente para a redução dos custos operacionais e para a minimização de penalidades por atraso na entrega das tarefas, como multas contratuais decorrentes de atrasos na partida de um navio. A sequência de recuperação das pilhas é influenciada pela taxa de recuperação das máquinas, pela data de entrega dos produtos e pela elegibilidade das recuperadoras para acessar pilhas específicas. Trata-se, sob essa ótica, de uma variante do problema de sequenciamento em máquinas paralelas não-relacionadas com tempos de preparação dependentes da sequência, datas de entrega e elegibilidade de máquinas, sendo classificado como NP-difícil.

O presente trabalho tem como objetivo *reproduzir* a pesquisa científica de Florinda, Souza e Cota [1], apresentada no LVI Simpósio Brasileiro de Pesquisa Operacional (SBPO 2024). Especificamente, busca-se: (i) compreender o problema e a modelagem propostos; (ii) reimplementar integralmente a formulação de programação linear inteira mista (MILP, do inglês *Mixed-Integer Linear Programming*) utilizando a API do resolvedor Gurobi em linguagem Python; e (iii) reproduzir os experimentos computacionais sobre as instâncias do problema, comparando os resultados obtidos com os reportados no trabalho original.

O restante deste relatório está organizado como segue. A Seção II apresenta um resumo do artigo reproduzido. A Seção III descreve detalhadamente a formulação matemática. A Seção IV detalha a implementação realizada e a Seção V descreve o ambiente computacional. A Seção VI reporta os resultados obtidos e a Seção VII os compara com os do artigo. Por fim, a Seção VIII apresenta as conclusões.

II. Resumo do Artigo Escolhido

O artigo reproduzido, intitulado "Uma formulação de programação linear inteira mista para o sequenciamento de recuperadoras em pátios de minério visando à minimização do atraso total" [1], trata do sequenciamento das operações de recuperação de pilhas de minério em pátios de estocagem. O problema é modelado como um problema de sequenciamento em máquinas paralelas não-relacionadas, no qual as tarefas representam as operações de recuperação de pilhas e as máquinas são as recuperadoras.

A principal contribuição do trabalho é preencher uma lacuna da literatura ao considerar *simultaneamente* características que, até então, eram tratadas de forma isolada: tempos de preparação (*setup*) dependentes da sequência, máquinas não-relacionadas (recuperadoras com diferentes taxas de recuperação), datas de entrega com penalidade por atraso, minimização da soma ponderada dos atrasos e elegibilidade de máquinas.

Para tanto, os autores propõem uma formulação MILP, a implementam no resolvedor Gurobi e a validam em um conjunto de instâncias de pequeno porte geradas a partir de dados operacionais do Porto de Tubarão (Vale S.A.), em Vitória-ES. Os resultados indicaram que o resolvedor foi capaz de encontrar a solução ótima para instâncias com até 10 tarefas e 4 recuperadoras, evidenciando ainda a necessidade de métodos meta-heurísticos para tratar instâncias de maior porte.

III. Descrição Detalhada da Formulação Matemática

Esta seção descreve a formulação MILP proposta em [1]. Apresentam-se, a seguir, os conjuntos, os índices, os parâmetros de entrada e as variáveis de decisão e auxiliares do modelo.

A. Conjuntos e Índices

- P: conjunto de pilhas a serem carregadas;
- $N = \{1, \ldots, |N|\}$: conjunto de tarefas de recuperação, em que cada tarefa $j \in N$ está associada a uma pilha $a \in P$;
- $N_0 = N \cup \{0\}$: conjunto de tarefas acrescido da *tarefa fictícia* 0, que precede a primeira tarefa de cada recuperadora;
- $RC = \{1, \ldots, |RC|\}$: conjunto de recuperadoras (máquinas);
- i: índice para uma recuperadora de RC; j, k: índices para tarefas de N (ou N_0); a: índice para uma pilha de P.

B. Parâmetros de Entrada

- q_j: quantidade de minério a ser recuperado na tarefa j (em toneladas);
- v_i: taxa de recuperação da recuperadora i (em t/h);
- $p_{ij} = q_j / v_i$: tempo de processamento da tarefa j na recuperadora i;
- S_{ijk}: tempo de preparação (*setup*) para a recuperadora i executar a tarefa k imediatamente após a tarefa j;
- e_{ij}: elegibilidade, igual a 1 se a recuperadora i pode processar a tarefa j e 0 caso contrário;
- B: constante suficientemente grande;
- d_j: data de entrega planejada da tarefa j;
- w_j: penalidade por atraso na entrega da tarefa j.

C. Variáveis de Decisão e Auxiliares

- $x_{ijk} \in \{0, 1\}$: assume valor 1 se a tarefa k é executada imediatamente após a tarefa j na recuperadora i;
- C_{ij}: tempo de conclusão da tarefa j na recuperadora i;
- C_j: tempo de conclusão da tarefa j;
- T_j: tempo de atraso da tarefa j, dado por $T_j = \max\{C_j - d_j, 0\}$.

D. Modelo Matemático

As expressões (1) a (12) representam a formulação MILP. A função objetivo (1) minimiza o atraso total ponderado:

$$\min\ z = \sum_{j=1}^{|N|} w_j\, T_j \tag{1}$$

sujeito a:

$$\sum_{i=1}^{|RC|} \sum_{\substack{j=0 \\ j \neq k \\ e_{ij}=1,\, e_{ik}=1}}^{|N|} x_{ijk} = 1 \qquad \forall k \in N \tag{2}$$

$$\sum_{i=1}^{|RC|} \sum_{\substack{k=1 \\ j \neq k \\ e_{ij}=1,\, e_{ik}=1}}^{|N|} x_{ijk} \leq 1 \qquad \forall j \in N \tag{3}$$

$$\sum_{\substack{k=1 \\ e_{ik}=1}}^{|N|} x_{i0k} \leq 1 \qquad \forall i \in RC \tag{4}$$

$$\sum_{\substack{h=0 \\ h \neq j,\, h \neq k \\ e_{ih}=1}}^{|N|} x_{ihj} \geq x_{ijk} \quad \forall j, k \in N,\, j \neq k,\, \forall i \in RC,\, e_{ij} = e_{ik} = 1 \tag{5}$$

$$C_{ik} + B\,(1 - x_{ijk}) \geq C_{ij} + S_{ijk} + p_{ik} \tag{6}$$
$$\forall j \in N_0,\, \forall k \in N,\, j \neq k,\, \forall i \in RC,\, e_{ij} = e_{ik} = 1$$

$$C_{i0} = 0 \qquad \forall i \in RC \tag{7}$$

$$C_j \geq C_{ij} \qquad \forall j \in N,\, \forall i \in RC,\, e_{ij} = 1 \tag{8}$$

$$T_j \geq C_j - d_j \qquad \forall j \in N \tag{9}$$

$$T_j \geq 0 \qquad \forall j \in N \tag{10}$$

$$C_{ij} \geq 0 \qquad \forall j \in N,\, \forall i \in RC,\, e_{ij} = 1 \tag{11}$$

$$x_{ijk} \in \{0, 1\} \qquad \forall j \in N_0,\, \forall k \in N,\, j \neq k,\, \forall i \in RC,\, e_{ij} = e_{ik} = 1 \tag{12}$$

A restrição (2) assegura que cada tarefa k seja designada exclusivamente a uma recuperadora elegível e tenha exatamente uma tarefa predecessora j (real ou fictícia) também elegível nessa recuperadora. A restrição (3) impõe um limite máximo de uma tarefa sucessora para cada tarefa j, desde que ambas sejam elegíveis na recuperadora. A restrição (4) estabelece um limite máximo de uma tarefa sucessora para a tarefa fictícia em cada recuperadora, isto é, cada recuperadora inicia no máximo uma sequência. A restrição (5) garante a conservação de fluxo: se j é predecessora imediata de k na recuperadora i, então j necessariamente possui uma predecessora imediata também elegível em i. A restrição (6) regula os tempos de conclusão: quando $x_{ijk} = 1$, o tempo de conclusão de k deve ser ao menos o tempo de conclusão de j somado ao *setup* S_{ijk} e ao processamento p_{ik}; quando $x_{ijk} = 0$, a constante B torna a restrição redundante. A restrição (7) fixa em zero o tempo de

conclusão das tarefas fictícias. A restrição (8) define o tempo de conclusão da tarefa j em uma recuperadora elegível. A restrição (9) define os tempos de atraso de cada tarefa. Por fim, as restrições (10), (11) e (12) definem os domínios das variáveis.

IV. Descrição da Implementação Realizada

A formulação apresentada na Seção III foi reimplementada em linguagem Python (versão 3.11), utilizando a API *gurobipy* do resolvedor Gurobi. Todo o código encontra-se no arquivo `solver.py` e cada elemento foi comentado com referência direta à equação correspondente do artigo, de modo a explicitar a correlação entre o código e a formulação.

A. Leitura das Instâncias

As instâncias do problema são armazenadas no arquivo `data/instances.csv`, no qual cada linha corresponde a uma instância. Os parâmetros de natureza vetorial ou matricial (q, v, d, e, S e w) são gravados como cadeias no formato JSON; assim, o código reconstrói as estruturas de dados por meio da função `json.loads`. A classe `Instance` encapsula o acesso aos parâmetros no estilo do artigo (indexação 1-based, com a tarefa fictícia 0), tratando dois detalhes de indexação relevantes: (i) a elegibilidade é armazenada como $e[j][i]$ (linha = tarefa, coluna = recuperadora), ao passo que o artigo usa e_{ij}; e (ii) o tempo de *setup* é armazenado como $S[i]$[anterior][próxima], com o índice da tarefa fictícia igual a 0.

B. Construção do Modelo

A função `build_model` constrói o modelo Gurobi. As variáveis binárias x_{ijk} são criadas apenas para pares de tarefas elegíveis ($e_{ij} = 1$ e $e_{ik} = 1$) e com $j \neq k$, reduzindo o número de variáveis conforme o domínio restrito das equações. As variáveis contínuas C_{ij}, C_j e T_j são criadas com limite inferior nulo (equações (10) e (11)). Cada restrição (2)–(9) é adicionada ao modelo por meio de `addConstr`, com um comentário indicando o número da equação. A constante B é calculada, por instância, como um limitante superior seguro para os tempos de conclusão: a soma, sobre todas as tarefas, do maior tempo de processamento possível, acrescida de $|N|$ vezes o maior tempo de *setup*.

C. Resolução e Relato

A função `solve_instance` configura os parâmetros do resolvedor (descritos na Seção V), invoca a otimização e coleta o valor da função objetivo (z), o melhor limite inferior (*best bound*), o *gap* de otimalidade e o tempo de execução. A função `main` percorre e resolve todas as instâncias do arquivo de entrada, imprimindo os resultados em um formato análogo ao da tabela de resultados do artigo.

V. Ambiente Computacional Utilizado

Os experimentos foram conduzidos em um notebook Acer Nitro 5, equipado com um processador Intel Core i5 de 11ª geração e 24 GB de memória RAM, sob sistema operacional Linux (distribuição Debian, *kernel* 6.1). A formulação foi implementada em Python 3.11, com as bibliotecas *gurobipy* (versão 11.0.2) e *pandas*. O resolvedor utilizado foi o Gurobi 11.0.2, operado sob licença acadêmica.

Em conformidade com o artigo original, foram adotados os seguintes parâmetros do resolvedor: tempo limite de 3600 segundos (`TimeLimit`) e foco na integralidade igual a 1 (`IntegralityFocus`), este último para assegurar que as variáveis binárias não sejam influenciadas por problemas de arredondamento durante a resolução. Como referência, o ambiente do trabalho original consistiu em um processador Intel Xeon de 2,20 GHz, 12 GB de memória RAM e sistema operacional Ubuntu 22.04.3 LTS.

VI. Resultados Obtidos

Para avaliar a formulação reimplementada, foram utilizadas 30 instâncias de pequeno porte, divididas em seis grupos definidos pelos pares $(|RC|, |N|)$: $(2, 8)$, $(2, 10)$, $(2, 12)$, $(4, 8)$, $(4, 10)$ e $(4, 12)$, com cinco instâncias por grupo. A Tabela I apresenta, para cada instância, o número de recuperadoras ($|RC|$), o número de tarefas ($|N|$), o valor da função objetivo (z), o melhor limite inferior (*best bound*), o *gap* de otimalidade e o tempo de execução, em segundos.

O resolvedor encontrou a solução ótima (*gap* de $0{,}00\%$) para todas as 30 instâncias. O tempo total de resolução do conjunto completo foi de aproximadamente 101 segundos, com tempo médio em torno de $3{,}4$ segundos por instância. A instância de maior esforço computacional foi a de número 14 (grupo $(2, 12)$), resolvida em $74{,}65$ segundos — ainda muito abaixo do limite de 3600 segundos.

Observa-se, ainda, que o esforço computacional cresce com o número de tarefas: enquanto as instâncias de 8 tarefas foram resolvidas em frações de segundo, o grupo de 12 tarefas concentrou os maiores tempos de execução (instâncias 14, 11 e 10). Nota-se também que, para um mesmo número de tarefas, as instâncias com apenas 2 recuperadoras tenderam a ser mais custosas do que as de 4 recuperadoras.

A título de ilustração do sequenciamento produzido, a Figura 1 apresenta o gráfico de Gantt da solução ótima da instância 8, à semelhança das Figuras 2 e 3 do artigo original. Cada linha corresponde a uma recuperadora; a parte hachurada representa o tempo de preparação (*setup*) e a parte colorida, o tempo de processamento de cada tarefa, com destaque em vermelho para as tarefas concluídas com atraso. Os tempos de conclusão são indicados no eixo das abscissas.

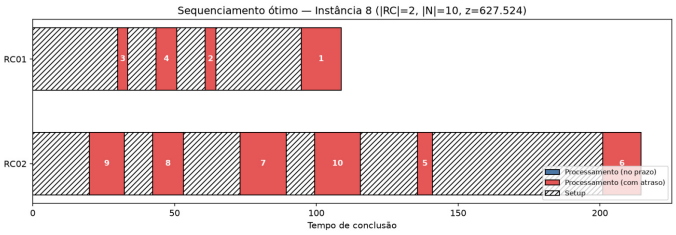


Figura 1. Sequenciamento ótimo da instância 8, com *setup* (hachurado) e processamento (colorido) por recuperadora.

Tabela I
RESULTADOS OBTIDOS PELA FORMULAÇÃO MILP NO RESOLVEDOR GUROBI.

ID	$\|RC\|$	$\|N\|$	z	*bound*	gap(%)	tempo(s)
1	2	8	484.671	484.671	0.00	0.12
2	2	8	581.596	581.596	0.00	0.15
3	2	8	409.825	409.825	0.00	0.30
4	2	8	412.011	412.011	0.00	0.13
5	2	8	562.190	562.190	0.00	0.20
6	2	10	640.993	640.993	0.00	0.35
7	2	10	779.438	779.438	0.00	0.61
8	2	10	627.524	627.524	0.00	0.59
9	2	10	528.084	528.084	0.00	0.43
10	2	10	550.478	550.478	0.00	5.77
11	2	12	687.396	687.396	0.00	6.83
12	2	12	927.286	927.286	0.00	2.37
13	2	12	847.090	847.090	0.00	1.48
14	2	12	961.804	961.804	0.00	74.65
15	2	12	983.919	983.919	0.00	2.30
16	4	8	370.513	370.513	0.00	0.06
17	4	8	645.588	645.588	0.00	0.06
18	4	8	451.635	451.635	0.00	0.19
19	4	8	459.667	459.667	0.00	0.08
20	4	8	543.775	543.775	0.00	0.09
21	4	10	509.621	509.621	0.00	0.07
22	4	10	412.908	412.908	0.00	0.07
23	4	10	667.266	667.240	0.00	0.07
24	4	10	588.841	588.841	0.00	0.07
25	4	10	525.633	525.633	0.00	0.33
26	4	12	839.160	839.160	0.00	1.24
27	4	12	691.174	691.174	0.00	0.08
28	4	12	907.227	907.227	0.00	1.21
29	4	12	569.364	569.364	0.00	0.15
30	4	12	754.428	754.428	0.00	0.74

VII. COMPARAÇÃO COM OS RESULTADOS DO ARTIGO

No trabalho original, o resolvedor Gurobi encontrou a solução ótima para 26 das 30 instâncias avaliadas. Para as instâncias 11, 12, 14 e 28 — todas com 12 tarefas —, o resolvedor não provou a otimalidade dentro do tempo limite de 3600 segundos, com *gaps* de 63,16%, 100%, 46,75% e 100%, respectivamente. Os autores concluem que o resolvedor foi capaz de resolver na otimalidade todas as instâncias com até 10 tarefas e 4 recuperadoras, evidenciando a necessidade de meta-heurísticas para instâncias maiores.

Ao comparar os resultados, observam-se dois aspectos. Em primeiro lugar, há uma **concordância qualitativa** quanto ao comportamento do modelo: em ambos os casos, o esforço computacional cresce com o número de tarefas, e o grupo de 12 tarefas é o mais desafiador; além disso, as instâncias com menos recuperadoras tendem a ser mais difíceis. Esse padrão confirma que a reimplementação captura corretamente a estrutura de dificuldade do problema.

Em segundo lugar, há uma **divergência numérica esperada**. Os valores de z obtidos não coincidem com os do artigo, e todas as 30 instâncias foram resolvidas na otimalidade em nossa execução, ao contrário das quatro instâncias não resolvidas no trabalho original. Essa diferença decorre do fato de as instâncias utilizadas terem sido *regeradas* localmente a partir do procedimento descrito no artigo (mesmos grupos e distribuições de probabilidade), e não corresponderem exatamente às instâncias originais disponibilizadas pelos autores no material suplementar. Diferenças nos valores sorteados de quantidade de minério, taxas de recuperação, datas de entrega, tempos de *setup* e penalidades produzem instâncias distintas —, em geral, de resolução mais fácil —, o que explica tanto a divergência nos valores de z quanto a ausência de estouros do tempo limite. Ressalta-se que nenhum aspecto da formulação foi alterado; a divergência é atribuível exclusivamente aos dados de entrada.

Cabe destacar que, na busca por reproduzir fielmente os experimentos, foi enviado um *e-mail* aos autores solicitando os dados exatos utilizados no trabalho original. Contudo, até a data de submissão deste artigo, não se havia obtido resposta. Ainda assim, foi possível replicar o estudo, uma vez que parte considerável dos dados de entrada pode ser gerada aleatoriamente segundo o procedimento e as distribuições de probabilidade descritos no próprio artigo, como foi feito neste trabalho. Dessa forma, a reprodução da metodologia e do comportamento do modelo permanece válida, ainda que as instâncias não sejam numericamente idênticas às originais.

VIII. CONCLUSÕES

Este trabalho reproduziu a formulação de programação linear inteira mista proposta por [1] para o problema de sequenciamento de recuperadoras em pátios de minério com minimização do atraso total ponderado. A formulação foi reimplementada com a API do resolvedor Gurobi em Python, com cada variável e restrição do código correlacionada às equações do artigo, e avaliada sobre um conjunto de 30 instâncias de pequeno porte.

Os experimentos demonstraram que a implementação reproduz fielmente a formulação original: o resolvedor encontrou a solução ótima para todas as instâncias avaliadas, e o comportamento qualitativo observado — dificuldade crescente com o número de tarefas e maior esforço para instâncias com menos recuperadoras — é consistente com o reportado no trabalho original. A única divergência, relativa aos valores numéricos de z e ao número de instâncias não resolvidas, decorre do uso de instâncias regeradas em vez das originais.

Como trabalhos futuros, destaca-se a reprodução numérica exata da tabela de resultados do artigo, mediante a obtenção das instâncias originais do material suplementar, bem como a exploração de métodos meta-heurísticos para tratar instâncias de maior porte, conforme apontado pelos próprios autores.

REFERÊNCIAS

[1] E. M. Florinda, M. J. F. Souza, and L. P. Cota, "Uma formulação de programação linear inteira mista para o sequenciamento de recuperadoras em pátios de minério visando à minimização do atraso total," em *Anais do LVI Simpósio Brasileiro de Pesquisa Operacional (SBPO)*, Fortaleza, CE, 2024. DOI: 10.59254/sbpo-2024-193585.

[2] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2024. [Online]. Disponível em: https://www.gurobi.com.

[3] J. D. F. Jesus-Junior, M. J. F. Souza, and L. P. Cota, "A hybrid metaheuristic algorithm for scheduling iron ore reclaiming in ports," *Engineering Optimization*, pp. 1–18, 2024. DOI: 10.1080/0305215X.2024.2340688.